



Instituto politécnico nacional

Escuela superior de computo



Unidad de aprendizaje:

Base de datos

Practica 1

"Modelo Entidad Relación"

Integrantes:

Lopez Teran Monserrat

Mata Lopez Jennifer

Grupo: 3CV4

Fecha de entrega: 18/09/2026

Ejercicio 1. Control de versiones con Git y GitHub

Parte A

¿Qué es un sistema de control de versiones?

Un sistema de control de versiones (VCS) rastrea cada alteración a un archivo o conjunto de archivos, lo que permite a los desarrolladores regresar a versiones anteriores y colaborar sin problemas. Los sistemas de control de versiones centralizados (CVC) optimizan este proceso al alojar todas las versiones de los archivos en un solo servidor. Los desarrolladores toman prestado un archivo para retocarlo y luego lo devuelven con actualizaciones, todo cuidadosamente almacenado y catalogado por el servidor. Este método es realmente muy simple, ya que ofrece una vía directa para gestionar los cambios.

Sin embargo, a medida que los equipos crecen y los proyectos se vuelven más intrincados, los sistemas de control de versiones distribuidos (DVCS), como Git, pasan a primer plano. DVCS no solo centraliza los archivos, sino que los democratiza. Cada desarrollador tiene todo el historial del proyecto a nivel local, lo que potencia el trabajo fuera de línea y facilita una variedad de estrategias de ramificación y fusión, esta flexibilidad es una bendición para los equipos dinámicos que buscan entrelazar múltiples hilos del proyecto sin enredarlos.

El control de versiones ya sea centralizado o distribuido, es esencial para el desarrollo de software eficiente y coherente, protege el progreso, aclara el pasado y allana el camino a seguir, al garantizar que cada miembro del equipo pueda contribuir con su mejor trabajo a la elaboración de un software excelente.

- Tipos de sistemas de control de versiones

Los dos tipos más populares de sistemas de control de versiones o revisiones son centralizado y distribuido, los sistemas de control de versiones centralizados almacenan todos los archivos en un repositorio central, mientras que los sistemas de control de versiones distribuidos almacenan archivos en varios repositorios, otros tipos menos comunes incluyen basado en bloqueos y optimista.

Distribuido

Un sistema de control de versiones distribuido (DVCS) permite a los usuarios acceder a un repositorio desde varias ubicaciones, por lo general, los desarrolladores que necesitan trabajar en proyectos desde varias computadoras o que necesitan colaborar con otros desarrolladores de forma remota utilizan los DVCS.

Centralizado

Un sistema de control de versiones centralizado (CVCS) es un tipo de VCS en el que todos los usuarios trabajan con el mismo repositorio central, este repositorio central puede ubicarse en un servidor o en la máquina local de un desarrollador, los sistemas de control de versiones centralizados se suelen utilizar en proyectos de desarrollo de software en los que un equipo de desarrolladores necesita compartir código y realizar un seguimiento de los cambios.

Basado en bloqueos

Un sistema de control de versiones basado en bloqueos utiliza el bloqueo de archivos para gestionar el acceso simultáneo a archivos y recursos, el bloqueo de archivos evita que dos o más usuarios realicen cambios en conflicto en el mismo archivo o recurso.

Optimista

En un sistema de control de versiones optimista, cada usuario tiene su propio espacio de trabajo privado, cuando alguien quiere compartir sus cambios con el resto del equipo, envía una solicitud al servidor, luego el servidor analiza todos los cambios y determina cuáles se pueden fusionar de manera segura.

Problema concreto que resuelve en un trabajo en equipo.

La sobreescritura accidental de datos y el caos de los cambios incompatibles, mejora de la calidad del código hasta la aceleración de los plazos de desarrollo y la mejora de la visibilidad del proyecto

Git y GitHub

- Git

Git es un sistema de control de versiones distribuido (SCVD) creado para ayudar a los desarrolladores a seguir y gestionar los cambios en su código, tanto si estás corrigiendo un error, añadiendo una nueva función o experimentando con una idea arriesgada, Git facilita el guardado de diferentes versiones de tu proyecto, para que siempre puedas volver atrás si algo se rompe, está diseñado pensando en la colaboración, cada desarrollador trabaja con una copia local completa del proyecto, lo que hace posible contribuir, confirmar y experimentar de forma independiente, una vez que estés listo, puedes sincronizar tus cambios con el resto del equipo a través de un repositorio remoto compartido.

Bifurcación - Trabaja en funciones de forma aislada sin alterar el código base principal.

Fusionar - Combina sin problemas los cambios de código de distintas ramas.

Commits - Guarda instantáneas de tu código, creando una línea de tiempo de tu progreso.

Seguimiento del historial - Mantén un registro detallado de cada cambio, por quién y por qué.

- GitHub

Si Git es el motor del control de versiones, GitHub es el garaje donde los equipos se reúnen para construir juntos. Se basa en las capacidades locales de Git ofreciendo una plataforma en la nube para alojar repositorios Git, convirtiendo los esfuerzos individuales en progreso colectivo, al almacenar tu código en una ubicación centralizada, GitHub permite a tu equipo acceder, revisar y contribuir al mismo proyecto, independientemente de dónde se encuentren.

Solicitudes de fusión: Proponer, revisar y discutir cambios en el código antes de fusionarlos.

Seguimiento de incidencias y tableros de proyectos: Gestiona errores, peticiones de funcionalidades y listas de tareas pendientes directamente en tu repositorio.

CI/CD con acciones de GitHub: Automatiza las pruebas, compilaciones y despliegues para mantener tu flujo de trabajo fluido y fiable.

Páginas GitHub: Aloja páginas estáticas directamente desde tu repositorio.

Herramientas de seguridad: Protege tu código con 2FA, escaneo de código y control de permisos.

GitHub Copiloto: Obtén sugerencias de programación en tiempo real basadas en IA para aumentar la productividad.

Diferencias de Git y GitHub

Función	Git	GitHub
Naturaleza	Sistema de control de versiones	Plataforma de alojamiento en la nube para repositorios Git
Ubicación	Funciona localmente en tu máquina	Requiere una conexión a Internet para acceder a los repositorios
Función principal	Rastrea los cambios en el código	Proporciona herramientas de colaboración y gestión de proyectos
Interfaz	Herramienta de línea de comandos (con opciones de GUI disponibles)	Interfaz gráfica basada en web
Propiedad	De código abierto, mantenido por la Fundación Linux	Propiedad de Microsoft
Elementos de seguridad	Carece de autenticación integrada	Admite autenticación de usuarios, control de acceso y permisos basados en funciones
Alternativas	Compite con SVN, Mercurial, Perforce	Compite con <u>GitLab</u> , <u>Bitbucket</u> , Azure DevOps

Repositorio

Un repositorio es un espacio de almacenamiento virtual donde se guardan los archivos de un proyecto y todo su historial de cambios, almacena no solo los archivos actuales (código, textos, imágenes), sino también quién hizo cada modificación, cuándo la hizo y por qué, puede ser local, en tu propia computadora o remoto en servidores en la nube como GitHub o GitLab

Commit

Un commit es una captura del estado de los archivos en un momento determinado, sirve como un punto de guardado al que podemos regresar si después se presenta algún problema o queremos recuperar una versión anterior, cada commit debe incluir un mensaje corto que indique qué cambios se realizaron, por ejemplo: "Se agregó la función de inicio de sesión", además, cada commit genera un código único llamado hash, que permite identificar y rastrear ese cambio posteriormente.

Rama (Branch)

En Git es una versión separada del proyecto que permite hacer cambios o probar cosas nuevas sin afectar directamente la versión principal, si tienes un proyecto de una página web y quieres agregar una función de inicio de sesión, puedes crear una rama llamada login, ahí haces todos los cambios y pruebas.

Cuando terminas y todo funciona correctamente, puedes unir esa rama con la principal (main).

En pocas palabras: una rama permite trabajar en cambios de manera independiente sin modificar directamente el proyecto principal.

fusión (merge)

La fusión (merge) en Git es cuando unes los cambios de una rama con otra. Normalmente se usa para pasar los cambios de una rama secundaria a la rama principal.

Imagina que tienes tu proyecto en una rama principal llamada main, en main tienes tu proyecto funcionando, creas una nueva rama llamada login para trabajar en el inicio de sesión, haces varios cambios y commits en esa rama, cuando terminas, tienes dos ramas:

main tiene el proyecto original.

login tiene el proyecto original más los cambios del inicio de sesión.

Haces un merge para unir los cambios de login con main.

Después del merge, main también tendrá la función de inicio de sesión.

Un conflicto de fusión (merge conflict)

Ocurre cuando Git intenta unir dos ramas, pero encuentra que las dos modificaron la misma parte de un archivo de diferentes maneras y no sabe cuál de los cambios debe conservar.

Ejemplo:

Supongamos que en main tenemos:

Nombre: Jennifer

Y en otra rama alguien cambia la misma línea

Nombre: Juan

Si intentas hacer un merge, Git no puede decidir si debe dejar Jennifer o Juan. Por eso genera un conflicto y te pide que tú decidas qué versión conservar, después de elegir y corregir el archivo, haces un nuevo commit para guardar la solución.

Un Pull Request (PR)

Es una solicitud para proponer que los cambios de una rama se agreguen a otra, normalmente a la rama principal (main).

Por ejemplo:

Tienes tu proyecto en main, creas una rama llamada login, en esa rama haces la función de inicio de sesión, cuando terminas, haces un Pull Request para pedir que tus cambios de login se integren en main, otra persona puede revisar tus cambios, hacer comentarios y, si todo está bien, aprobarlos y hacer el merge.

Un Pull Request es una solicitud para revisar y aceptar los cambios de una rama antes de unirlos con otra.

El archivo .gitignore

Es un archivo que se utiliza para indicarle a Git qué archivos o carpetas no debe tomar en cuenta ni guardar en el repositorio.

Por ejemplo, en un proyecto pueden existir archivos que no queremos subir, como archivos temporales, contraseñas, configuraciones personales o carpetas creadas automáticamente.

Primero creamos un archivo llamado .gitignore:

```
.gitignore
```

Dentro de ese archivo escribimos lo que quieres que Git ignore:

```
node_modules/
```

```
.env
```

```
*.log
```

Entonces quedaría:

```
mi-proyecto/
```

```
.gitignore
```

```
index.html
```

El archivo .gitignore es el que contiene las instrucciones de qué archivos o carpetas Git debe ignorar.

En qué consiste un flujo de trabajo basado en ramas y por qué el código se revisa entre pares antes de fusionarse.

Un flujo de trabajo basado en ramas (como Git Flow o GitHub Flow) consiste en crear una línea de desarrollo independiente de la línea principal (usualmente llamada main o master) para trabajar en una nueva característica, corrección de errores o experimento sin alterar el código que ya está funcionando en producción.

El proceso técnico se compone de cuatro pasos principales:

Creación: El desarrollador crea una copia exacta de la rama principal en una nueva rama secundaria.

Desarrollo aislado: Se realizan los cambios, pruebas y confirmaciones (commits) exclusivamente en esa rama.

Solicitud de fusión: Una vez terminado el trabajo, se abre una solicitud de revisión (conocida como Pull Request o Merge Request).

Fusión: Tras ser aprobada, la rama secundaria se integra de vuelta a la rama principal y luego se elimina.

¿Por qué el código se revisa entre pares antes de fusionarse?

La revisión por pares (Code Review) es un control de calidad fundamental porque previene errores en producción, un compañero puede detectar fallos de lógica, vulnerabilidades de seguridad o casos extremos que al autor se le pasaron por alto, asegura la legibilidad y consistencia, garantiza que el código siga los estándares del equipo, lo que facilita que cualquier miembro pueda mantenerlo o modificarlo en el futuro, comparte el conocimiento, todo el equipo se mantiene al tanto de los cambios que se introducen en el sistema, evitando que una sola persona sea la única que entienda cómo funciona una parte del software, fomenta el aprendizaje continuo, es un espacio para sugerir mejores prácticas, optimizaciones de rendimiento y técnicas de programación más eficientes entre compañeros.

Ejercicio 2. El sistema gestor en un contenedor: Docker

PARTE A

Qué es un contenedor

Los contenedores son una forma más ligera y ágil de gestionar la virtualización, ya que no utilizan un hipervisor, lo que permite disfrutar de un aprovisionamiento de recursos más rápido y una disponibilidad más rápida de nuevas aplicaciones, en lugar de poner en marcha una máquina virtual completa, la contenedorización agrupa todo lo necesario para ejecutar una sola aplicación o microservicio (junto con las bibliotecas de tiempo de ejecución que necesitan para funcionar), el contenedor incluye todo el código, sus dependencias e incluso el propio sistema operativo, esto permite que las aplicaciones se ejecuten prácticamente en cualquier lugar: una computadora de escritorio, una infraestructura de TI tradicional o la nube.

Los contenedores utilizan una forma de virtualización del sistema operativo (SO), en pocas palabras, aprovechan las características del sistema operativo anfitrión para aislar los procesos y controlar su acceso a las CPU, la memoria y el espacio de escritorio.

Los contenedores existen desde hace décadas, sin embargo, el consenso general es que la era moderna de los contenedores comenzó en 2013 con la introducción de Docker, una plataforma de código abierto para crear, desplegar y gestionar aplicaciones contenedorizadas.

Diferencia de una máquina virtual y un contenedor

En la virtualización tradicional, un hipervisor virtualiza el hardware físico, el resultado es que cada máquina virtual contiene un SO invitado, una copia virtual del hardware que el SO necesita para funcionar y una aplicación con sus bibliotecas y dependencias asociadas, se pueden ejecutar VM con diferentes sistemas operativos en el mismo servidor físico, por ejemplo, una VM VMware puede ejecutarse junto a una VM Linux, que a su vez se ejecuta junto a una VM Microsoft, etc.

En lugar de virtualizar el hardware subyacente, los contenedores virtualizan el sistema operativo (normalmente, Linux) de manera que cada contenedor individual contiene solo la aplicación y sus bibliotecas y dependencias, los contenedores son pequeños, rápidos y portátiles porque, a diferencia de una máquina virtual, no necesitan incluir un SO invitado en cada instancia y, en su lugar, pueden simplemente aprovechar las características y los recursos del SO anfitrión.

Al igual que las máquinas virtuales, los contenedores permiten a los desarrolladores mejorar la utilización de la CPU y la memoria de las máquinas físicas, sin embargo, los contenedores van aún más allá, ya que también permiten arquitecturas de microservicios, en las que los componentes de las aplicaciones se pueden desplegar y escalar de forma más granular, esta es una alternativa atractiva a la ampliación de una aplicación monolítica completa porque un solo componente está lidiando con su carga.

Característica	Contenedor	Máquina virtual
Arranque	Muy rápido, normalmente segundos o menos	Más lento, porque inicia un sistema operativo completo
Tamaño	Pequeño, puede ser de MB o algunos GB	Mucho más grande, normalmente varios GB
Aislamiento	Aísla aplicaciones, pero comparte el kernel del sistema anfitrión	Aislamiento más fuerte: cada VM tiene su propio sistema operativo y kernel

Imagen

Una imagen es una plantilla que contiene todo lo necesario para crear y ejecutar un contenedor: la aplicación, sus dependencias, archivos y configuración.

Contenedor

Un contenedor es una instancia en ejecución de una imagen. Es un entorno aislado donde se ejecuta una aplicación.

Volumen

Un volumen es un espacio de almacenamiento que permite guardar datos de un contenedor de manera independiente

Puerto publicado

Un puerto publicado permite que un servicio que está funcionando dentro de un contenedor pueda ser accesible desde fuera del contenedor, por ejemplo, desde un navegador.

¿Para qué sirve el volumen?

Un volumen es un mecanismo de almacenamiento utilizado por Docker para conservar los datos generados o utilizados por un contenedor de forma independiente a su ciclo de vida, esto permite que la información permanezca disponible aunque el contenedor sea eliminado

¿Por qué es indispensable?

El volumen es indispensable cuando una aplicación necesita conservar información de manera permanente, como bases de datos, archivos o configuraciones, si no se utiliza un volumen, los datos almacenados dentro del contenedor pueden perderse al eliminarlo.

Ejercicio 3. Investigación: ¿qué es una base de datos?

Dato: Es un hecho, valor, número, palabra u observación que puede ser registrado y posteriormente procesado para obtener información, los datos son un conjunto de hechos, números, palabras, observaciones u otra información útil, a través del procesamiento de datos y análisis de datos, las organizaciones transforman datos sin procesar en insights valiosos que mejoran la toma de decisiones e impulsan mejores resultados comerciales.

Las organizaciones recopilan datos de diversas fuentes y en varios formatos, incluidos datos cualitativos no numéricos (como comentarios de clientes) y datos cuantitativos numéricos (como cifras de ventas), otros ejemplos de datos incluyen datos públicos, como estadísticas del gobierno y registros del censo, y datos privados, como historiales de compras de clientes o registros de atención médica de una persona.

Durante la última década, el big data (grandes y complejos conjuntos de datos provenientes de fuentes como las redes sociales, el comercio electrónico y las transacciones financieras) ha impulsado la transformación digital en todas las industrias. De hecho, el big data se ganó el apodo de "el nuevo petróleo" por su valor como impulsor del crecimiento y la innovación empresarial.

Datos cuantitativos: Los datos cuantitativos están conformados por valores que se pueden medir numéricamente. Los ejemplos de datos cuantitativos incluyen puntos de datos discretos (como la cantidad de productos vendidos) o puntos de datos continuos (como la temperatura o las cifras de ingresos).

Datos cualitativos: son descriptivos y no numéricos, ya que capturan características, conceptos o experiencias que los números no pueden medir. Algunos ejemplos son el feedback de clientes, comentarios de productos y comentarios en redes sociales.

Información: Es el conjunto de datos que han sido recopilados, organizados, procesados e interpretados de manera que adquieren un significado determinado, la información permite comprender mejor una situación, identificar características o relaciones entre diferentes elementos y puede utilizarse como apoyo para la toma de decisiones, a diferencia de un dato aislado, la información proporciona un contexto que permite comprender qué representan esos datos y cuál es su utilidad

Elmasri y Navathe	C. J. Date
Se enfocan en los datos relacionados.	Se enfoca en los datos utilizados por una organización.
Destacan que representa una parte del mundo real.	Destaca su utilización por sistemas de aplicación.
Consideran que debe tener coherencia y un propósito específico.	Destaca su carácter operacional.

Bases de datos: Una base de datos es un repositorio digital para almacenar, gestionar y proteger colecciones organizadas de datos, los diferentes tipos de bases de datos almacenan datos de diferentes maneras, por ejemplo, las bases de datos relacionales lo almacenan en tablas definidas con filas y columnas, mientras que las bases de datos no relacionales pueden almacenarlo como una variedad de estructuras de datos, incluidos pares clave-valor o gráficos.

Las organizaciones utilizan estos diferentes tipos de bases de datos para gestionar diferentes tipos de datos, las bases de datos relacionales sobresalen con datos estructurados, como registros financieros, las bases de datos no relacionales son mejores para tipos de datos no estructurados, como archivos de texto, audio y video., las bases de datos vectoriales almacenan datos como incrustaciones vectoriales, un formato utilizado por muchas aplicaciones de IA generativa.

Tipos de bases de datos:

- Bases de datos de navegación
- Bases de datos relacionales
- Bases de datos no relacionales o bases de datos NoSQL
- Bases de datos orientadas a objetos
- Bases de datos vectoriales
- Bases de datos en la nube

Característica de una base de datos

Una base de datos debe contar con ciertas características que permiten que los datos sean almacenados y utilizados de manera organizada, segura y eficiente.

Integración

La integración consiste en reunir y relacionar los datos de diferentes áreas o procesos dentro de un mismo sistema, evitando que la información se encuentre aislada en diferentes archivos.

Persistencia

La persistencia es la capacidad de una base de datos para conservar los datos almacenados durante un periodo prolongado, de manera que puedan ser consultados posteriormente, incluso después de cerrar el programa o apagar el equipo.

Redundancia controlada

La redundancia controlada consiste en evitar la duplicación innecesaria de datos, aunque en determinadas situaciones puede permitirse cierta duplicación cuando resulta útil para mejorar el funcionamiento del sistema.

Integridad

La integridad garantiza que los datos almacenados sean correctos, válidos y coherentes, evitando que se introduzca información incorrecta o que se generen relaciones inválidas entre los datos.

Independencia de datos

La independencia de datos es la capacidad de modificar la estructura o la forma en que se almacenan los datos sin tener que modificar necesariamente los programas que utilizan esos datos.

Seguridad

La seguridad consiste en proteger la base de datos contra accesos, modificaciones o eliminaciones no autorizadas. Para ello se pueden establecer usuarios, contraseñas, permisos y diferentes niveles de acceso.

Concurrencia

La concurrencia es la capacidad de una base de datos para permitir que varios usuarios o aplicaciones accedan y trabajen con los datos al mismo tiempo, manteniendo la consistencia de la información.

Recuperación

La recuperación es la capacidad de una base de datos para restaurar la información y mantener su consistencia después de una falla, como un error del sistema, una interrupción eléctrica o algún problema con el almacenamiento.

Archivos vs bases de datos

¿Qué es el enfoque tradicional de archivos?

Antes de utilizar sistemas gestores de bases de datos, era común que cada aplicación almacenara la información en archivos independientes, cada programa tenía sus propios archivos y estaba diseñado específicamente para trabajar con ellos.

Redundancia de datos

La redundancia ocurre cuando el mismo dato se almacena varias veces en diferentes archivos.

Esto puede provocar:

- Mayor uso de espacio.
- Mayor trabajo para actualizar los datos.
- Mayor posibilidad de que las diferentes copias terminen siendo diferentes.

Elmasri y Navathe señalan que almacenar los mismos datos varias veces provoca desperdicio de espacio y duplicación del esfuerzo de mantenimiento

Inconsistencia de datos

La inconsistencia aparece cuando existen varias copias del mismo dato, pero alguna de ellas se modifica y las demás no.

Silberschatz, Korth y Sudarshan explican que la duplicación de información en diferentes archivos puede provocar inconsistencias cuando una copia se actualiza y las demás no.

Dependencia programa-datos

La dependencia programa-datos es una característica del enfoque tradicional de archivos en la que los programas dependen de la estructura y organización específica de los archivos que utilizan.

En otras palabras, el programa necesita conocer cómo están organizados los datos para poder leerlos, modificarlos o procesarlos, por esta razón, un cambio en la estructura de un archivo puede obligar a modificar también los programas que trabajan con él, esta dependencia dificulta el mantenimiento de los sistemas, ya que los cambios en los datos pueden producir cambios en los programas.

Diferencia entre redundancia e inconsistencia

Redundancia es la duplicación de datos, inconsistencia es cuando esas copias dejan de coincidir

Aislamiento de datos

Los datos pueden encontrarse distribuidos en diferentes archivos y formatos, dificultando su combinación y consulta.

Dificultad para compartir información

Cuando cada aplicación administra sus propios archivos, compartir los datos entre diferentes aplicaciones puede ser complicado, por ejemplo, el programa de inscripciones puede tener información sobre los alumnos que también necesita el programa de biblioteca, si cada uno mantiene sus propios archivos, se puede terminar duplicando información.

Problemas de integridad

La integridad de los datos consiste en mantener la información correcta, válida y coherente, en un sistema de archivos, las reglas que garantizan la validez de los datos suelen estar implementadas dentro de los programas.

Por ejemplo, un programa puede impedir que una calificación sea mayor que 10, pero otro programa podría no tener esa misma restricción, esto puede provocar que diferentes aplicaciones manejen los datos de manera diferente.

Problemas de seguridad

Cuando existen muchos archivos administrados por diferentes aplicaciones, puede ser más difícil establecer y controlar de manera centralizada quién puede consultar, modificar o eliminar determinada información, un sistema de bases de datos proporciona mecanismos específicos para administrar usuarios y permisos.

Problemas de acceso simultáneo

Cuando varios usuarios intentan trabajar con los mismos datos al mismo tiempo, pueden producirse conflictos si no existen mecanismos adecuados para controlar las operaciones, por ejemplo, dos usuarios podrían intentar modificar simultáneamente el mismo registro.

Los SGBD incorporan mecanismos de control de concurrencia para administrar este tipo de situaciones.

¿Por qué el enfoque de archivos dio origen a los SGBD?

Con el crecimiento de las organizaciones aumentó también la cantidad de información que necesitaban almacenar y procesar.

El enfoque de archivos comenzó a presentar dificultades porque:

- Los datos podían estar repetidos.
- Las copias podían volverse inconsistentes.
- Los programas dependían de la estructura de los archivos.
- Era difícil compartir información entre aplicaciones.
- Las consultas nuevas podían requerir nuevos programas.
- El control de integridad podía ser complicado.
- El acceso simultáneo podía generar problemas.
- La seguridad era difícil de administrar de manera centralizada.
- Por estas razones surgió el Sistema Gestor de Bases de Datos (SGBD).

Un SGBD es un software que permite crear, almacenar, organizar, consultar, modificar, proteger y administrar bases de datos, en lugar de que cada programa tenga que administrar directamente sus propios archivos, existe un sistema especializado que funciona como intermediario entre las aplicaciones y los datos.

¿Qué solucionan las bases de datos?

El cambio del enfoque de archivos al enfoque de bases de datos permitió desarrollar mecanismos para:

- Reducir y controlar la redundancia:
Los datos pueden organizarse para evitar duplicaciones innecesarias.
- Mantener la consistencia:
Las modificaciones pueden realizarse sobre una fuente común de datos.
- Separar los programas de los datos:
Se busca que los cambios en la forma de almacenar los datos no obliguen a modificar todos los programas.
- Compartir información:
Varias aplicaciones pueden utilizar los mismos datos.
- Controlar la seguridad:
Se pueden establecer usuarios, permisos y niveles de acceso.
- Controlar la concurrencia:
Varios usuarios pueden trabajar con los datos de manera controlada.

- Mantener la integridad:
El SGBD puede establecer restricciones para evitar datos inválidos.
- Recuperar información después de fallos:
Los SGBD proporcionan mecanismos de recuperación para proteger la información ante determinados errores o fallas.

Usuarios de una base de datos

Un usuario de base de datos se define como una persona que interactúa diariamente con los datos para actualizarlos, leerlos y modificarlos. Los usuarios de base de datos pueden acceder a los datos y recuperarlos a través de las aplicaciones e interfaces del Sistema de Gestión de Bases de Datos (DBMS).

Diseñadores de bases de datos

Los diseñadores de bases de datos crean la estructura de una base de datos, incluyendo tablas, vistas, índices, disparadores y restricciones, deciden qué datos almacenar y cómo relacionarlos según los requisitos del usuario, su diseño se finaliza antes de que la base de datos se construya o se llene con datos.

Administrador de bases de datos (DBA)

Un administrador de bases de datos (DBA) define el esquema y gestiona los tres niveles de la arquitectura de la base de datos. Crea cuentas de usuario, controla el acceso y garantiza la seguridad de los datos autorizando únicamente a usuarios de confianza

Los DBA también son responsables de gestionar las brechas de seguridad y supervisa la recuperación y las copias de seguridad, y proporciona soporte técnico.

Tiene una cuenta de DBA en el sistema de gestión de bases de datos (DBMS), que se denomina cuenta de sistema o de super usuario.

Repara los daños causados por fallos de hardware y/o software.

Es quien tiene privilegios para realizar operaciones DCL (Lenguaje de Control de Datos), como GRANT y REVOKE , para permitir o restringir el acceso de un usuario específico a la base de datos.

Programadores de aplicaciones

Los programadores de aplicaciones, también conocidos como analistas de sistemas o ingenieros de software, escriben el código de backend para los programas de aplicación, utilizan lenguajes como C, Visual Basic, COBOL, etc., para diseñar, probar y mantener los programas, estos programas, denominados "transacciones predefinidas", ayudan a los usuarios inexpertos a interactuar con la base de datos de forma eficiente.

Usuarios finales

Los usuarios finales paramétricos son personas sin conocimientos avanzados de sistemas de gestión de bases de datos (DBMS), pero que utilizan frecuentemente estas aplicaciones en su vida diaria para obtener los resultados deseados.

Analista de sistemas

Un analista de sistemas es un usuario que analiza los requisitos de los usuarios finales paramétricos, verifica si se cumplen todos los requisitos de los usuarios finales.

Ciclo de vida de una base de datos

El ciclo de vida de una base de datos es el conjunto de etapas que se siguen para planificar, analizar, diseñar, implementar, probar, utilizar y mantener una base de datos durante su existencia, este proceso permite desarrollar la base de datos de manera organizada, comenzando con la identificación de las necesidades de los usuarios y continuando con el mantenimiento y actualización del sistema una vez que se encuentra en funcionamiento.

El ciclo de vida puede dividirse en diferentes etapas dependiendo de la metodología o autor que se utilice, algunas metodologías agrupan varias actividades en una misma etapa, mientras que otras las separan para describir el proceso con mayor detalle, por ejemplo, la Open University contempla actividades relacionadas con requisitos, análisis, diseño, implementación, pruebas y mantenimiento, mientras que el Internal Revenue Service (IRS) utiliza cinco grandes etapas para describir el Database Life Cycle: análisis de requisitos, diseño lógico, diseño físico, implementación y monitoreo, modificación y mantenimiento (The Open University, 2016; Internal Revenue Service, 2024).

Para explicar el proceso de manera detallada, el ciclo de vida de una base de datos puede dividirse en las siguientes etapas: planificación, definición del sistema, recolección y análisis de requisitos, diseño conceptual, diseño lógico, diseño físico, implementación, pruebas y operación, monitoreo y mantenimiento.

1. Planificación

La planificación es la etapa inicial del ciclo de vida, en ella se determina la necesidad de desarrollar una base de datos y se establecen los objetivos generales que se desean alcanzar, también se analiza el problema que se pretende solucionar, el alcance inicial del proyecto, los recursos disponibles y las personas que participarán en su desarrollo.

Esta etapa permite determinar qué se quiere conseguir antes de comenzar con el diseño técnico de la base de datos, una planificación adecuada ayuda a establecer una dirección para el proyecto y permite identificar desde el inicio las necesidades generales de la organización.

Por ejemplo, una universidad puede detectar que actualmente administra la información de sus alumnos mediante diferentes archivos y programas. Debido a esto, decide desarrollar una base de datos que permita administrar alumnos, materias, profesores, grupos e inscripciones.

El entregable principal de esta etapa es un plan o propuesta del proyecto, en el que se establecen los objetivos, el alcance inicial, los recursos necesarios y las necesidades generales de la base de datos.

2. Definición del sistema

En esta etapa se determina con mayor claridad qué parte de la organización será administrada por la base de datos y cuáles serán los límites del sistema, también se identifican los principales usuarios, las áreas involucradas y las funciones generales que deberá realizar el sistema.

La definición del sistema es importante porque permite establecer qué información será responsabilidad de la base de datos y qué información quedará fuera de su alcance. De esta manera se evita intentar solucionar problemas que no corresponden al proyecto.

Por ejemplo, si se desarrolla una base de datos para administrar una universidad, puede establecerse que el sistema manejará alumnos, profesores, materias, grupos y calificaciones, pero que la información de nómina de empleados será administrada por otro sistema.

El entregable de esta etapa es la definición y documentación del alcance del sistema, incluyendo los objetivos, límites, usuarios y funciones generales.

3. Recolección y análisis de requisitos

La recolección y análisis de requisitos consiste en identificar las necesidades de los usuarios y determinar qué debe hacer la base de datos, para obtener esta información pueden utilizarse entrevistas, cuestionarios, observación de los procesos existentes y análisis de documentos.

En esta etapa se identifican los datos que deben almacenarse, las operaciones que deberán realizarse, las consultas que los usuarios necesitarán, los informes que deberán generarse, las reglas de negocio, las restricciones y los requisitos de seguridad.

La Open University (2016) señala que la determinación de requisitos implica consultar a las partes interesadas para establecer qué necesitan del sistema y llegar a un acuerdo sobre sus necesidades.

Por ejemplo, al entrevistar al personal administrativo de una universidad se puede determinar que cada alumno debe tener una matrícula única, que un alumno puede inscribirse en varias materias y que cada materia puede tener muchos alumnos.

El entregable principal de esta etapa es el documento o especificación de requisitos, que contiene las necesidades de información, funciones, reglas de negocio, restricciones y otros requisitos que deberá cumplir la base de datos.

4. Diseño conceptual

El diseño conceptual consiste en representar de manera general los datos que deberán almacenarse y las relaciones que existen entre ellos, sin preocuparse todavía por la forma específica en que serán almacenados físicamente.

En esta etapa se construye un modelo conceptual de los datos, uno de los modelos más utilizados es el modelo entidad-relación, mediante el cual se identifican entidades, atributos y relaciones.

Por ejemplo, para una universidad se pueden identificar entidades como ALUMNO, MATERIA, PROFESOR y GRUPO, posteriormente se establecen las relaciones entre ellas, un alumno puede estar relacionado con varias materias y una materia puede estar relacionada con varios alumnos.

El diseño conceptual permite representar las necesidades de información de la organización de una manera independiente de un sistema gestor de bases de datos específico.

El entregable principal es el modelo conceptual de datos, normalmente representado mediante un diagrama entidad-relación acompañado de la descripción de las entidades, atributos, relaciones y restricciones.

5. Diseño lógico

El diseño lógico transforma el modelo conceptual en una estructura que puede ser implementada mediante un modelo de datos determinado, en el caso de las bases de datos relacionales, el diseño conceptual se transforma en tablas, atributos, claves y relaciones.

Durante esta etapa se determinan las tablas que formarán parte de la base de datos, los atributos de cada tabla, las claves primarias, las claves foráneas y las restricciones necesarias para mantener la integridad de los datos.

Por ejemplo, una entidad ALUMNO puede convertirse en una tabla con los atributos matrícula, nombre, correo y carrera, la matrícula puede utilizarse como clave primaria para identificar de manera única a cada alumno, el diseño lógico permite establecer cómo se organizarán los datos desde el punto de vista lógico antes de decidir cómo serán almacenados físicamente.

El entregable principal es el esquema lógico de la base de datos, que incluye las tablas, atributos, claves, relaciones y restricciones.

6. Diseño físico

El diseño físico consiste en determinar cómo se almacenará y organizará físicamente la base de datos en un entorno tecnológico determinado, en esta etapa se consideran las características del sistema gestor de bases de datos, el almacenamiento disponible y los requerimientos de rendimiento.

Entre las decisiones del diseño físico pueden encontrarse la creación de índices, la organización del almacenamiento, la distribución de los datos y diferentes configuraciones destinadas a mejorar el rendimiento y la seguridad.

Por ejemplo, si una universidad realiza miles de consultas utilizando la matrícula del alumno, se puede crear un índice sobre ese campo para facilitar y acelerar las búsquedas, el diseño físico depende del sistema gestor de bases de datos que se utilizará y del entorno de hardware y software donde funcionará la base de datos (The Open University, 2016).

El entregable de esta etapa es la especificación del diseño físico, que describe las estructuras de almacenamiento, índices, configuraciones y demás decisiones técnicas necesarias para implementar la base de datos.

7. Implementación

La implementación consiste en convertir los diseños anteriores en una base de datos funcional utilizando un sistema gestor de bases de datos (SGBD), en esta etapa se crean las tablas, claves, relaciones, restricciones, índices, vistas, usuarios y permisos, también se pueden cargar los datos iniciales y desarrollar las aplicaciones que utilizarán la base de datos.

Por ejemplo, utilizando PostgreSQL se puede crear una tabla para almacenar la información de los alumnos mediante una instrucción SQL:

```
CREATE TABLE alumno (  
matricula INT PRIMARY KEY,  
nombre VARCHAR(100),  
carrera VARCHAR(100)  
);
```

Después de ejecutar esta instrucción, la tabla deja de ser solamente parte del diseño y pasa a existir dentro de la base de datos.

El entregable principal de esta etapa es la base de datos implementada y funcional, junto con las configuraciones, aplicaciones y documentación necesarias.

8. Pruebas

Las pruebas tienen como objetivo comprobar que la base de datos funciona correctamente y que cumple con los requisitos establecidos durante las etapas anteriores, durante esta etapa se pueden realizar pruebas sobre las consultas, inserciones, modificaciones y eliminaciones de información, también se comprueba el funcionamiento de las restricciones de integridad, los permisos de los usuarios, la seguridad, el rendimiento y otros aspectos relevantes.

La Open University (2016) distingue entre verificación y validación. La verificación permite comprobar que el sistema se ha construido correctamente de acuerdo con sus especificaciones, mientras que la validación permite determinar si el sistema satisface las necesidades para las que fue desarrollado.

Por ejemplo, si las calificaciones permitidas están entre 0 y 10, se puede intentar registrar una calificación de 15 para comprobar que la restricción correspondiente funcione correctamente.

El entregable principal de esta etapa es el informe o registro de pruebas, donde se documentan las pruebas realizadas, sus resultados, los errores encontrados y las correcciones necesarias.

9. Operación, monitoreo y mantenimiento

Después de haber sido implementada y probada, la base de datos entra en la etapa de operación, sin embargo, el ciclo de vida no termina cuando la base de datos comienza a utilizarse, durante esta etapa se realiza el monitoreo y mantenimiento del sistema. Se pueden corregir errores, actualizar información, mejorar el rendimiento, modificar estructuras, agregar nuevas funciones, realizar respaldos y actualizar las medidas de seguridad.

El mantenimiento también permite adaptar la base de datos cuando cambian las necesidades de los usuarios o de la organización, por ejemplo, si una universidad decide comenzar a registrar la modalidad de las materias, será necesario realizar modificaciones en el sistema para almacenar esta nueva información.

La Open University (2016) señala que el mantenimiento puede incluir correcciones, cambios derivados de nuevos requisitos y adaptaciones a nuevos entornos, por su parte, el Internal Revenue Service (2024) considera el monitoreo, la modificación y el mantenimiento como parte del ciclo de vida posterior a la implementación.

Por lo tanto, el ciclo de vida de una base de datos es un proceso que puede repetirse cuando aparecen nuevos requisitos o problemas importantes, es posible regresar a etapas anteriores, realizar modificaciones y volver a probar el sistema.

El entregable de esta etapa puede incluir registros de mantenimiento, informes de monitoreo, documentación actualizada, respaldos y versiones modificadas de la base de datos.

Sistema gestor de bases de datos

Sistema Gestor de Bases de Datos (SGBD) es un software que permite crear, administrar, proteger y utilizar bases de datos de forma centralizada y segura.

Módulos componentes

- Compilador DDL: Traduce las instrucciones de definición de estructura y las guarda en el diccionario de datos.
- Precompilador DML: Procesa las órdenes de manipulación de datos para integrarlas con lenguajes de programación principales.
- Procesador de consultas: Traduce las peticiones de búsqueda a instrucciones de bajo nivel eficientes.
- Gestor de archivos y almacenamiento: Controla la interacción con el sistema operativo para guardar los datos en el disco físico.

- Gestor de transacciones y planificador: Asegura la consistencia, integridad y aislamiento de las operaciones simultáneas (propiedades ACID).
- Diccionario de datos: Repositorio central con los metadatos o la descripción de la estructura de la base de datos.

Lenguajes del SGBD

- DDL (Data Definition Language): Define la estructura y el esquema de la base de datos. Comandos comunes: CREATE, ALTER, DROP.
- DML (Data Manipulation Language): Permite consultar, insertar, modificar y eliminar los datos almacenados. Comandos comunes: SELECT, INSERT, UPDATE, DELETE.
- DCL (Data Control Language): Administra la seguridad, el acceso y los privilegios de los usuarios. Comandos comunes: GRANT, REVOKE.
- Control de transacciones (TCL): Administra los cambios hechos por las transacciones para garantizar la fiabilidad del sistema. Comandos comunes: COMMIT, ROLLBACK

Clasificación de los SGBD

- Según el modelo de datos: Relacionales (SQL), No Relacionales (NoSQL), Orientados a Objeto.
- Según el número de usuarios: Monousuario (un solo usuario a la vez) y Multiusuario (concurrentes).
- Según la distribución: Centralizados (en una sola máquina) y Distribuidos (repartidos en varios servidores o ubicaciones de red).

Sistema de base de datos

Arquitectura de 3 niveles (Ansi/sparc)

La arquitectura de tres niveles también conocida como ANSI-SPARC identificó la necesidad de un enfoque de tres niveles, donde los tres niveles de abstracción comprenden un nivel externo, uno conceptual y uno interno. La arquitectura de tres niveles busca separar la vista que cada usuario tiene de la base de datos de la forma en que esta se representa físicamente.

Nivel externo: Es la vista que tiene el usuario de la base de datos. En este nivel se describen los datos relevantes para el usuario. El nivel externo consta de varias vistas externas diferentes de la base de datos. En la vista externa solo se incluyen las entidades, atributos y relaciones que el usuario necesita. Las diferentes vistas pueden representar los mismos datos de distintas maneras. Por ejemplo, un usuario puede ver el nombre en el formato (nombre, apellido), mientras que otro puede verlo como (apellido, nombre).

Nivel conceptual: Representa la visión de la comunidad sobre la base de datos y describe qué datos se almacenan en ella, incluyendo las entidades, sus atributos y sus relaciones. También representa la información semántica, de seguridad e integridad de los datos. El nivel conceptual, o segundo nivel en la arquitectura de tres niveles, contiene la estructura lógica de toda la base de datos y representa la visión completa que la organización requiere, independientemente de cualquier consideración de almacenamiento.

Nivel interno: A nivel interno, la base de datos se representa físicamente en el ordenador. Se hace hincapié en la implementación física de la base de datos para optimizar el uso del espacio de almacenamiento, lograr un rendimiento óptimo en tiempo de ejecución y aplicar técnicas de cifrado de datos. Interactúa con el sistema operativo para almacenar los datos en archivos, crear el espacio de almacenamiento, recuperar los datos, etc.

Independencia de datos

La independencia de datos es un concepto fundamental en los sistemas de gestión de bases de datos (DBMS) que se refiere a la capacidad de modificar el esquema en un nivel de la base de datos sin afectar el esquema en el nivel superior. Este concepto garantiza que los cambios en la forma en que se almacenan o estructuran internamente los datos no afecten la manera en que los usuarios o las aplicaciones acceden a ellos e interactúan con ellos.

¿Por qué es importante la independencia de los datos?

- Reduce el mantenimiento: Los desarrolladores no necesitan actualizar las aplicaciones cada vez que cambia la estructura de la base de datos.
- Mayor flexibilidad: La base de datos se puede reorganizar u optimizar internamente sin afectar las consultas de los usuarios.
- Favorece el crecimiento a largo plazo: a medida que evolucionan las necesidades del negocio, la base de datos se puede actualizar sin afectar a los sistemas existentes.

Dentro de la arquitectura de 3 niveles tenemos

Existe independencia de datos entre estos niveles:

- Independencia de datos físicos: Cambio en el nivel interno sin afectar el nivel lógico.
- Independencia lógica de los datos: Cambio en el nivel lógico sin afectar el nivel de vista.

Independencia lógica de los datos

Permite modificar la estructura lógica (tablas, columnas, relaciones) sin afectar las vistas externas ni los programas de aplicación. Su propósito es permitir que la estructura de la base de datos evolucione sin afectar el acceso de los usuarios ni requerir cambios en el código de la aplicación.

Por qué es importante:

Ayuda a modificar la estructura de la base de datos a medida que evolucionan las necesidades del negocio.

Las aplicaciones y las interfaces de usuario no se verán afectadas.

Ejemplo:

- Agregar una columna como correo electrónico en la tabla de empleados.
- Crear una nueva relación entre dos tablas.
- Combinar dos tablas en una vista para facilitar el acceso.

Independencia de datos físicos

La capacidad de modificar la forma en que se almacenan físicamente los datos sin afectar el esquema lógico ni las aplicaciones de cara al usuario. El objetivo es mejorar el rendimiento, la eficiencia del almacenamiento o las configuraciones de hardware sin alterar la estructura lógica de los datos.

Por qué es importante:

Permite la optimización del rendimiento y los cambios de hardware.

No afecta a cómo se estructuran o consultan los datos.

Ejemplo:

- Transferencia de archivos de datos de la unidad C: a la unidad D:.
- Crear un índice para acelerar las consultas.
- Cambiar de HDD a SSD para un mejor rendimiento.
- Comprimir archivos de datos para ahorrar espacio.

Arquitectura Centralizada

Una base de datos centralizada es aquella que está totalmente en un solo lugar físico, es decir, está almacenada en una sola máquina y en una sola CPU, en la cual los usuarios trabajan en terminales que solo muestran resultados.

Los sistemas de bases de datos centralizadas son aquellos que se ejecutan en un único sistema informático sin interactuar con ninguna otra computadora. Tales sistemas van desde los sistemas de bases de datos mono usuarios ejecutándose en computadoras personales hasta los sistemas de bases de datos de alto rendimiento encuitándose en grandes sistemas.

Características de las bases de datos centralizadas.

- Se almacena completamente en una localidad central.
- No posee múltiples elementos de procesamiento ni mecanismos de intercomunicación como las bases de datos distribuidas.
- Los componentes de las bases de datos centralizadas son: los datos, el software de gestión de bases de datos y los dispositivos de almacenamiento secundario asociados.

Ventajas de las bases de datos centralizadas.

- Se evita la inconsistencia.
- Pueden aplicarse restricciones de seguridad.
- El procesamiento de los datos ofrece un mejor rendimiento y resulta más confiable que los sistemas distribuidos.

Desventajas de las bases de datos centralizadas.

- Si el sistema de base de datos falla, se pierde la disponibilidad y procesamiento de la información que posee el sistema.
- Dificil sincronización para su recuperación.

Arquitectura cliente servidor

La arquitectura cliente-servidor es un modelo informático que divide las tareas o cargas de trabajo entre los proveedores de servicios, denominados servidores, y los solicitantes de servicios, denominados clientes. Este modelo está diseñado para mejorar la eficiencia y la gestión de los recursos centralizando las funciones del servidor, que se encarga del procesamiento y la gestión de los datos, mientras que los clientes interactúan con el usuario y solicitan servicios al servidor.

El modelo de arquitectura cliente-servidor consta de dos componentes principales: el cliente y el servidor.

Cliente

El cliente es un ordenador o una aplicación de software que inicia las solicitudes al servidor. Los clientes suelen estar orientados al usuario y proporcionan la interfaz a través de la cual los usuarios interactúan con el servidor. Algunos ejemplos de clientes son los navegadores web, los clientes de correo electrónico y las aplicaciones móviles.

Servidor

Un servidor es un ordenador potente o una aplicación de software que proporciona servicios a los clientes a través de una red. Los servidores alojan, gestionan y distribuyen recursos como datos, aplicaciones y capacidad de procesamiento. Algunos ejemplos comunes de servidores son los servidores web, los servidores de bases de datos y los servidores de archivos.

Ventajas de la arquitectura cliente-servidor

- La arquitectura cliente-servidor ofrece numerosas ventajas, lo que la convierte en una opción popular para diversas aplicaciones y servicios. Entre estos beneficios se incluyen:
- Administración centralizada : La administración centralizada es una de las ventajas más importantes de la arquitectura cliente-servidor. Al consolidar el control de los datos, las aplicaciones y la seguridad en el servidor, los administradores pueden garantizar la coherencia en toda la red. Esta centralización simplifica tareas como

las actualizaciones, las copias de seguridad y la resolución de problemas, lo que se traduce en un mantenimiento más eficiente y una menor inactividad.

- Escalabilidad : La arquitectura cliente-servidor es altamente escalable, lo que permite a las empresas crecer sin grandes interrupciones. Los servidores se pueden actualizar con hardware más potente o recursos adicionales para gestionar un número creciente de solicitudes de clientes. Esta flexibilidad permite ampliar los servicios y la capacidad de usuarios de forma gradual, garantizando que el sistema pueda satisfacer las demandas cambiantes.
- Compartición de recursos : Una de las principales ventajas de esta arquitectura es la compartición eficiente de recursos. Los servidores pueden proporcionar datos, aplicaciones y capacidad de procesamiento a múltiples clientes simultáneamente. Esta utilización eficiente de los recursos reduce la redundancia y los costos, ya que no es necesario replicar los recursos en varios dispositivos.
- Seguridad mejorada : Los servidores centralizados permiten implementar medidas de seguridad robustas para proteger la información confidencial. Los administradores pueden implementar protocolos de seguridad integrales, supervisar el acceso y aplicar políticas de forma coherente. Esto reduce el riesgo de filtraciones de datos y accesos no autorizados, garantizando la seguridad de la información confidencial.

Modelos de datos

Un modelo de datos es un plano estructurado que define cómo se organizan, almacenan, representan y relacionan los datos dentro de un sistema de información

Importancia de los modelos de datos

- Organización lógica: Estructuran la información de forma clara y coherente para evitar la redundancia.
- Alineación con el negocio: Traducen los requerimientos de una empresa en reglas y flujos de trabajo funcionales.
- Independencia de datos: Separan el diseño de la aplicación de cómo se guardan físicamente los datos en el hardware o software.
- Integridad y seguridad: Establecen restricciones que protegen la precisión y consistencia de la información

Clasificación por nivel de abstracción (Arquitectura ANSI/SPARC)

Se dividen en tres etapas esenciales del diseño:

Modelo Conceptual (Nivel Alto):

- Ofrece una visión global del negocio.
- Define las entidades principales (ej. Cliente, Producto) y sus relaciones generales.
- Ignora cualquier detalle técnico o de implementación

Modelo Lógico (Nivel Intermedio):

- Detalla los atributos de cada entidad y el tipo de relación exacta (uno a uno, uno a muchos).
- No depende de un sistema de gestión de bases de datos (SGBD) en específico, pero ya define una estructura formal.

Modelo Físico (Nivel Bajo):

- Especifica cómo se almacenarán los datos en un SGBD concreto (ej. MySQL, Oracle, MongoDB).
- Define tipos de datos exactos, longitudes, índices y claves primarias o foráneas.
- También incluye el nivel externo o de vistas, que representa cómo ve la información cada usuario o departamento específico.

Clasificación por estructura (Modelos de bases de datos)

Describe cómo se interconectan lógicamente los registros:

Modelo Relacional:

- Organiza los datos en tablas (filas y columnas) conectadas por claves compartidas.
- Es el estándar para sistemas transaccionales

Modelo Jerárquico:

- Ordena los datos en una estructura de árbol con una relación de padres a hijos (un solo padre por cada hijo).

Modelo de Red:

- Similar al jerárquico, pero permite que un registro hijo tenga múltiples padres (estructura de grafo), lo que reduce la redundancia.

Modelo Orientado a Objetos:

- Almacena los datos en forma de objetos (con propiedades y métodos), integrándose directamente con la programación orientada a objetos.

Modelo NoSQL (No relacional):

- Utiliza esquemas flexibles como documentos (JSON), pares clave-valor, o grafos.
- Es ideal para grandes volúmenes de datos no estructurados y análisis en tiempo real.

Referencias:

- GitLab. (s. f.). ¿Qué es el control de versiones? GitLab. <https://about.gitlab.com/es/topics/version-control/>
- Internal Revenue Service. (2024). Database design techniques and deliverables. U.S. Department of the Treasury. https://www.irs.gov/irm/part2/irm_02-005-013
- Teorey, T. J., Lightstone, S. S., Nadeau, T., & Jagadish, H. V. (2011). Database modeling and design: Logical design (5th ed.). Morgan Kaufmann.
- The Open University. (2016). The database development life cycle. OpenLearn. <https://www.open.edu/openlearn/science-maths-technology/the-database-development-life-cycle/content-section-0>
- OpenTextBC. (n.d.). Chapter 13: Database development process. In Database Design – 2nd Edition. <https://opentextbc.ca/dbdesign01/chapter/chapter-13-database-development-process/>
- Silberschatz; Korth, Sudarshan, Fundamentos de Bases de Datos; Quinta Edición; McGraw
-
- GeeksforGeeks. (2020, 13 de febrero). The three-level ANSI-SPARC architecture. GeeksforGeeks
- José D. (2012, noviembre). 5. *Arquitectura centralizada*. Base de Datos. normalizacion-bd.blogspot.com
- Super Micro Computer, Inc. (s. f.). ¿Qué es la arquitectura cliente-servidor? Supermicro. supermicro.com
- Vide, O. (2024, 17 de septiembre). *Modelos de datos: Fundamentos para una arquitectura de información sólida*. Incentro. incentro.com
- DataCamp. (2025, 19 de septiembre). *Explicación de los modelos de datos DBMS: Tipos, niveles y ejemplos de PostgreSQL*. DataCamp. datacamp.com

- Jeremiah, O. (2025, 10 de abril). *Git frente a GitHub: Diferencias que todo desarrollador debe conocer*. DataCamp. <https://www.datacamp.com/es/blog/git-vs-github>
- IBM. (s. f.). *Contenedores frente a máquinas virtuales (VM): ¿Cuál es la diferencia?* <https://www.ibm.com/mx-es/think/topics/containers-vs-vms>
- Badman, A., & Kosinski, M. (s. f.). *¿Qué son los datos?* IBM. <https://www.ibm.com/mx-es/think/topics/data>
- IBM. (s. f.). *¿Qué es una base de datos?* <https://www.ibm.com/mx-es/think/topics/database>
- Elmasri, R., & Navathe, S. B. (2007). *Fundamentos de sistemas de bases de datos* (5.ª ed.). Pearson Educación.
- Watt, A. (2014). Characteristics and benefits of a database. En *Database design* (2nd ed.). BCcampus. <https://opentextbc.ca/dbdesign01/chapter/chapter-3-characteristics-and-benefits-of-a-database/>
- GeeksforGeeks. (2025, 2 de diciembre). *Different types of database users*. <https://www.geeksforgeeks.org/dbms/different-types-of-database-users/>